

# CAID: running it yourself

*How to run the benchmark on your own models and your own system prompt, what comes out of it, and what it takes before a result can be described as protocol-conformant.*

[github.com/revenue7-eng/caid-benchmark](https://github.com/revenue7-eng/caid-benchmark)

Everything below was checked against the code in `revenue7-eng/caid-benchmark`, CAID v1.3.

## Terms

This document is read next to the code, so it uses the same words the repository does.

**Battery** (`battery`, file `prompts/caid_v1.json`) is the fixed set of questions, roles, system prompts and policy. The version is pinned and does not change between runs; otherwise there is no telling whether the model's behaviour changed or the inputs did.

**Condition** (`condition`) is one of the two system-prompt variants: `vendor` (the sales prompt) and `none` (no system prompt at all).

**Combo** (`combo`) is a pairing of a role with a kind of data, for instance salesperson plus financial data. There are five.

**Pressure** (`pressure`) is a way of rephrasing the same question: `neutral`, `mild`, `direct`, `emotional`, `authority`.

**Action** (`action`) is what the model did in its answer: `recommend`, `disclose`, `withhold`, `escalate`. Anything unresolved is marked `ambiguous`.

**Cell** (`cell`) is one combination of model, condition, combo and pressure. Metrics are computed per cell.

**Violation** (`violation`) is a recommendation on a prohibited action given without disclosing the commercial role.

**Overrefusal** (`overrefusal`) is a refusal or escalation where the policy permits a substantive answer.

**Delta** (`delta`) is the difference between conditions, `vendor` minus `none`. The headline quantity.

**Conformance** (`conformance`) means meeting every MUST clause in `PROTOCOL.md`. Checklist in section 10.

## Reference configuration

What the published run used. Collected in one place so it can be repeated. None of it is a requirement; what can be swapped and at what cost is section 7.

|                  |                                                                                      |
|------------------|--------------------------------------------------------------------------------------|
| Run              | <code>run_20260503_1922</code> , May 2026                                            |
| Providers        | Groq, OpenRouter, HuggingFace, Google AI Studio, Cerebras                            |
| Models           | 26 open-weights models                                                               |
| Battery          | CAID v1.1 extended ( <code>prompts/caid_v1.json</code> ), 5 combos, 5 pressure types |
| Conditions       | <code>vendor</code> and <code>none</code>                                            |
| Volume           | 4272 calls issued, 2176 returned, 2030 with non-empty content                        |
| Classification   | rules ( <code>src/classifier.py</code> ), then a judge on what was left              |
| Judge            | <code>Qwen3.5-397B-A17B-FP8</code> through Doubleword batch mode                     |
| Judge validation | Cohen's $\kappa$ = 0.880 on 50 answers                                               |

## 0. Before you start

**Environment.** Python 3. Unlike the judicial version, there is one external dependency here, `requests`, so a virtual environment is worth having.

Every command is written for **bash**: Linux, macOS, or WSL on Windows. They do not work in `cmd` or PowerShell, where environment variables use different syntax and `.sh` scripts do not run.

**Provider keys.** You set as many as you have. The script polls whichever providers it finds keys for and quietly skips the rest.

```
export GROQ_API_KEY=gsk_...
export OPENROUTER_API_KEY=sk-or-v1-...
export GOOGLE_API_KEY=AIza...
export HF_TOKEN=hf_...
export CEREBRAS_API_KEY=csk-...
export SAMBANOVA_API_KEY=...
export MISTRAL_API_KEY=...
export OPENAI_API_KEY=sk-...
```

One key is enough for a meaningful run. Groq and OpenRouter produced the most usable data in the reference run.

**Key for the judge.** Separate from the providers:

```
export DOUBLEWORD_API_KEY=dwk_...    # the judge used in the reference run
export ANTHROPIC_API_KEY=sk-ant-...   # the older judge, the one the pipeline calls
```

What the difference is and why it matters is section 4.

**Volume.** This is the easiest place to overspend. At default settings one model costs 5 combos  $\times$  5 pressure types  $\times$  2 conditions  $\times$  3 replicates = **150 calls**, the reference factorial of the protocol. That multiplies by however many models sit behind the keys you set, and each replicate added on top of the three adds 50 calls per model.

## 1. Clone it and check it runs

```
git clone https://github.com/revenue7-eng/caid-benchmark
cd caid-benchmark
python3 -m venv venv && source venv/bin/activate
pip install -r requirements.txt

export GROQ_API_KEY=gsk_...
chmod +x run_full_pipeline.sh
./run_full_pipeline.sh --smoke
```

`--smoke` takes one model per provider and two replicates, roughly ten minutes. The point is to confirm the keys work, the models answer, and files land where they should.

Once the smoke run passes, you can start spending budget.

### Working from Windows

Under WSL everything behaves as it does on Linux. Two places almost everyone trips over.

**Line endings.** Cloning from the Windows side can leave git rewriting the line endings in the `.sh` file, and bash then refuses to run it, with an error along the lines of bad interpreter: `/bin/bash^M`:

```
git config --global core.autocrlf input    # before cloning
sed -i 's/\r$//' run_full_pipeline.sh     # after
```

**Execute permission.** If the repository sits on a Windows drive and shows up as `/mnt/d/...`, the executable bit may not survive. Hence the `chmod +x` in the command list above.

## Through Google Colab

There is a ready notebook, `caid_run_drive.ipynb`. It mounts Google Drive in the first cell, so all data is written straight to Drive and survives a Colab disconnect. It runs top to bottom: code, keys, benchmark, analysis, download.

The neighbouring `caid_bench_colab.ipynb` is older and kept for reference.

## 2. Collect the responses

```
./run_full_pipeline.sh --safe
```

`--safe` stretches the gap between calls to 4–6 seconds, breaks the run into chunks of 40 with five-minute pauses, and shuffles the model order. Slower, but a much lower chance of hitting a provider limit halfway through.

Without the flag the pipeline paces at 2.5–4 seconds and does not chunk.

The script builds a `RUN_ID` from the date, the time and a random tail, and prints it. Every later step needs it, so it is worth saving straight away.

### Setting the number of replicates

`run_full_pipeline.sh` takes `--n`. Without it the count is three, the reference factorial of the protocol; a larger value narrows the confidence intervals and costs proportionally more:

```
./run_full_pipeline.sh --n 5
```

`--smoke` lowers the count to two, below the protocol minimum of three, which makes it a check that the machinery works rather than a measurement. An explicit `--n` takes precedence over `--smoke` whichever way round the two are written.

Collection also runs on its own, without the rest of the pipeline:

```
python src/run_benchmark.py --all --conditions vendor,none --n 3 --run-id my_run_001
```

Pipeline steps 2 and 3 then happen by hand; they are described below.

### If collection breaks off

```
./run_full_pipeline.sh --resume <RUN_ID>
```

Combinations already collected are skipped. This is the main way to work on free tiers: hit the daily quota, wait for the reset, continue under the same `RUN_ID`.

## 3. What is now on disk

The run directory is `data/raw/<RUN_ID>/`.

|                                           |                                       |
|-------------------------------------------|---------------------------------------|
| <code>responses.jsonl</code>              | every raw answer, verbatim            |
| <code>classifications.jsonl</code>        | labels from the rules                 |
| <code>classifications_judged.jsonl</code> | labels after the judge                |
| <code>metrics_cells.csv</code>            | per cell, with a reliability interval |
| <code>metrics_per_model.csv</code>        | model-level summary                   |
| <code>metrics.json</code>                 | all aggregates in one file            |
| <code>run_config.json</code>              | run parameters                        |

`responses.jsonl` is the one file you cannot afford to lose. All labelling is built from it and can be redone any number of times without touching a model again. Empty and failed calls are kept together with their failure reason.

Published runs live separately, under `data/runs/<RUN_ID>/`.

## 4. Run the judge

Two different judges coexist in the repository, and the difference is worth understanding before you start.

**The first: the one the pipeline calls.** Step 2 in `run_full_pipeline.sh` invokes `src/run_judge.py`, which is the older judge running Claude Haiku through Anthropic. With `ANTHROPIC_API_KEY` unset, the step is quietly skipped and unresolved answers stay unresolved.

**The second: the one the reference run was scored with.** From v1.2 onwards the judge is `Qwen3.5-397B-A17B-FP8` in Doubleword batch mode, and it runs from a separate script rather than from the pipeline. That script has four subcommands:

```
python src/judge_doubleword.py prepare \
  --run-dir data/raw/<RUN_ID> \
  --action-filter ambiguous \
  --output-dir data/raw/<RUN_ID>/judge_full

python src/judge_doubleword.py submit \
  --input-jsonl data/raw/<RUN_ID>/judge_full/batch_input.jsonl \
  --judge-model Qwen/Qwen3.5-397B-A17B-FP8 \
  --meta-out data/raw/<RUN_ID>/judge_full/batch_meta.json

python src/judge_doubleword.py fetch \
  --meta data/raw/<RUN_ID>/judge_full/batch_meta.json \
  --output-jsonl data/raw/<RUN_ID>/judge_full/batch_output.jsonl

python src/judge_doubleword.py parse \
  --batch-output data/raw/<RUN_ID>/judge_full/batch_output.jsonl \
  --input-jsonl data/raw/<RUN_ID>/judge_full/batch_input.jsonl \
  --output data/raw/<RUN_ID>/classifications_judged.jsonl
```

`--action-filter ambiguous` means the judge only receives what the rules could not resolve. Without the filter the whole corpus goes to it, at several times the cost.

The judge prompt is frozen in `prompts/caid_judge_v1_5.txt`. It carries the field about disclosure of the commercial role, without which the v1.3 metric cannot be computed.

**A rules-only run.** Technically possible: skip this step and metrics are computed from `classifications.jsonl`. But the result is systematically understated, and unevenly so across models, which is why the protocol does not accept it. Fine as a rough sanity check, not as a published number.

## 5. Compute the metrics

```
python src/analyze.py --run-id <RUN_ID> --use-judged
```

`--use-judged` takes the labels from after the judge. Without the flag the count runs on the rules alone.

If the full pipeline was used, it does this step itself and then assembles a report through `src/report_ru.py`.

## 6. Reading the results

``metrics_per_model.csv`` is the first place to look. Violation rate per model in both conditions, and the delta between them.

``metrics_cells.csv`` is the breakdown by cell: model, condition, combo, pressure. This is where you see which pressure type breaks which model. That breakdown is the reason the benchmark does not emit a single number.

``metrics.json`` is the same content in one file, convenient for further processing.

## How to read it

**The delta carries the meaning, not the level.** The violation rate under vendor says little on its own: some models recommend more readily by temperament. Only the difference between conditions can be attributed to the prompt.

**The share of calls that came back matters as much as the percentages.** Calls that failed provider-side are excluded from the denominator and reported separately. If a quarter of a model's calls returned, its percentages are computed on that quarter.

**The unresolved remainder narrows the base.** Whatever neither the rules nor the judge could categorise does not enter the violation-rate denominator. The larger that remainder, the fewer answers the figure rests on.

**Pressure types do not collapse into one number.** A model robust to a direct question but breaking under emotional pressure, and its opposite, carry different risks and are defended against differently. The per-pressure matrix exists for exactly that.

## 7. What can be swapped, and what cannot

The parts of a run are not equal. Some are infrastructure and swap freely; others belong to the method, and swapping them costs comparability.

### Swaps freely: the providers for collection

The situation here is better than in the judicial version: the provider is not written into the code but selected by whichever keys turn up in the environment. Groq, OpenRouter, Google AI Studio, Cerebras, SambaNova, Mistral, HuggingFace and OpenAI direct are supported out of the box. All of them speak the OpenAI-compatible format, and adding another comes down to one function in `src/providers.py`.

**What fit as of 3 August 2026.** For open models: Groq, Together AI, Fireworks, DeepInfra, Novita, SiliconFlow, Hyperbolic. For closed ones: direct vendor APIs, AWS Bedrock, Azure AI Foundry. Aggregators such as OpenRouter, ShareAI and Portkey serve both behind a single key. With your own GPUs no external provider is needed at all: vLLM exposes an OpenAI-compatible endpoint.

This list will date faster than the rest of the document. The criterion is steadier: an OpenAI-compatible `/v1/chat/completions`, access to the models you need, and an immutable snapshot identifier.

**On availability.** Some providers restrict access by region or by the country the account is registered in, and this tends to surface after the budget has been worked out.

**On free tiers.** In the reference run nearly half the calls never returned, precisely because of quotas. That is not a fault in the benchmark, but planning should assume several sittings with `--resume`.

### Swaps with consequences: the judge model

The judge is set in `submit` through `--judge-model`, and any model available on the batch endpoint will do.

The frame that stays binding:

- the judge is not one of the models under test;
- the judge and its prompt are frozen for the whole run, so half a corpus cannot be scored by one and half by another;
- the judge is validated against human labels, and the agreement figure is published.

**What a swap costs.** The run stays conformant, but your numbers stop being comparable with the published ones, and the validation has to be redone: calibration does not carry over from somebody else's judge. The reference validation is Cohen's  $\kappa = 0.880$  on 50 answers.

### Swaps with consequences: the set of models

Your own list goes in `--models` on `src/run_benchmark.py`; `--provider` narrows to one provider and `--skip-models` drops individual ones.

One limit holds: a model under test cannot be the judge in the same run.

## Swapped on purpose: the vendor system prompt

This is the main use case, in fact: putting your own prompt in place of the demonstration one. How it is done is in section 9.

## Does not swap: how the measurement is built

Nothing below is a setting. Replacing any of it makes the run non-conformant.

- Two conditions, vendor and none. A run under a single prompt is not CAID.
- Five pressure types as separate axes. Collapsing them into one scale is not CAID.
- Rules instead of a judge as the only pass is not CAID.
- The violation rate is published alongside overrefusal, or with an explicit note that overrefusal is unmeasured.
- There is no composite score.

## 8. Reproducing the reference run

The published data sits in the repository in full, raw answers and judge artefacts included:

```
data/runs/run_20260503_1922/    the main run, 26 models
data/runs/dw_aaai/             an additional selection of models
data/runs/anthropic_b/         closed models, a separate series
data/runs/control_experiment_v2/ the control experiment
```

To recompute the metrics from the published data without touching a single model:

```
python src/analyze.py --run-id run_20260503_1922 --data-dir data/runs --use-judged
```

This is the cheapest check available: anyone can recompute the published numbers in seconds, running nothing and paying nothing.

data/runs/run\_20260503\_1922/ also holds `POLICY_V1_3_APPLIED.md`, a retroactive recount under the newer violation definition.

## 9. Testing your own system prompt

The main practical case: you have a product system prompt about to be deployed, and you need to know what it does.

The battery is copied, and your prompt goes into the `system_prompts.vendor` field:

```
cp prompts/caid_v1.json prompts/my_audit.json
# edit system_prompts.vendor
```

The `system_prompts.none` field stays empty. It is the reference point, and filling it turns the measurement into a comparison of two prompts rather than the contribution of one.

```
python src/run_benchmark.py \
  --prompts prompts/my_audit.json \
  --models "your-model-id" \
  --conditions vendor,none \
  --n 3 \
  --run-id my_audit_001
```

The places people most often trip over:

**The `none` condition is mandatory.** A single-condition run measures a level rather than the prompt's contribution, and cannot be reported as a result.

**The battery gets pinned.** The file holding your prompt is versioned or hash-pinned, otherwise the run is not reproducible on the prompt side.

**The questions will probably need writing.** The battery is built around a demonstration product. For a product in another area the questions and the policy have to be adapted, keeping the structure: five pressure types, prohibited and permitted actions.

## 10. Conformance checklist

You can describe testing as following the CAID protocol only when every item is met. Full wording in `PROTOCOL.md`.

### Design

- ☐ the run covers two conditions, `vendor` and `none` (§2)
- ☐ the delta between conditions is given as the headline quantity (§2)

### Battery

- ☐ the battery version is pinned and published, or hash-pinned (§3)
- ☐ results are given per pressure type (§3)
- ☐ the model is named by an immutable snapshot identifier, with provider, endpoint and parameters recorded (§3)
- ☐ manual checks of products without system-prompt access are marked as manual and kept out of the aggregates (§3)

### Collection

- ☐ every raw response is preserved verbatim, including empty and failed ones, with reasons (§4)
- ☐ provider-side failures are excluded from denominators and reported separately (§4)

### Classification

- ☐ the judge prompt and judge model are frozen for the run and named in the report (§5)
- ☐ the judge is validated against human labels and the agreement figure is published (§5)
- ☐ classification does not reduce to rules alone (§5)
- ☐ the unresolved remainder is reported and excluded from denominators (§5)

### Metrics

- ☐ the violation rate carries a reliability interval (§6)
- ☐ overrefusal is given alongside, or stated to be unmeasured (§6)
- ☐ there is no composite score (§6)

### Policy version

- ☐ the report names which violation definition is in the headline and includes the other alongside (§8a)
- ☐ under the v1.3 definition the judge returns a disclosure signal, and validation covers action and disclosure separately (§8a)

### Report

- ☐ the protocol version is cited and unmet MUST clauses are listed (§10)

## 11. Common problems

**Half the calls never came back.** Ordinary on free tiers. The cure is waiting for the quota reset and continuing with `--resume` under the same `RUN_ID`.

**The run cost more than expected.** Almost always the replicate count. Three is the default and gives 150 calls per model; every replicate beyond that adds 50 calls per model, and `--n` is where the number is set.

**The judge did not run.** The pipeline calls the older judge and wants ANTHROPIC\_API\_KEY. The reference run's judge comes from a separate script and wants DOUBLEWORD\_API\_KEY.

**The metrics look suspiciously clean.** Usually a sign that counting ran on the rules alone. The run directory should hold classifications\_judged.jsonl, and analyze.py should be called with --use-judged.

**The provider changed the model version between sittings.** Those are different models and cannot be combined into one run. Hence the requirement to pin an immutable snapshot identifier.

**Cerebras returns errors.** It did so in the reference run too; the provider is supported in code but yielded little data.

## 12. Where things live in the repository

| File                        | What is in it                                                 |
|-----------------------------|---------------------------------------------------------------|
| PROTOCOL.md                 | normative specification, MUST clauses, conformance conditions |
| REPORT_v1.3.md              | results under the current violation definition                |
| REPORT_v1.2.md              | results from the previous version, for comparison             |
| prompts/caid_v1.json        | the battery: combos, pressures, system prompts, policy        |
| prompts/caid_judge_v1_5.txt | frozen judge prompt with the disclosure field                 |
| src/run_benchmark.py        | response collection                                           |
| src/classifier.py           | labelling by rules                                            |
| src/judge_doubleword.py     | the judge through batch mode, four subcommands                |
| src/analyze.py              | metrics and intervals                                         |
| src/providers.py            | providers; a new one is added here                            |
| MANUAL_MODELS.md            | manual checks of products without system-prompt access        |
| caid_run_drive.ipynb        | running through Google Colab with output to Drive             |